

PROJECT DOMAIN: HEALTHCARE

Heart Disease Prediction

Using Ridge- and Lasso-Regularized Logistic Regression

TEAM MEMBERS

K ANUSRI ~ 2520030432

J SIMRAN ~ 2520030166

Problem Definition



Why it matters

Cardiovascular disease remains one of the leading causes of death worldwide. Predicting risk early gives patients and doctors a critical window to act before a crisis occurs.



What problem are we solving?

Predicting the likelihood that a patient has heart disease from routine clinical measurements, using a regularized logistic regression model.



Why is this important?

Early, low-cost risk screening can flag high-risk patients sooner, enabling timely referral, lifestyle change, or treatment.



Who will use the solution?

Doctors, clinicians, and healthcare screening programs — as a decision-support aid, not a replacement for diagnosis.



What is the expected output?

A predicted class (heart disease present / absent) with an associated risk probability for each patient record.

Dataset Collection

Building a reliable clinical dataset before any modelling begins.

- 1 Source a public clinical dataset — e.g. the UCI Machine Learning Repository / Kaggle Heart Disease (Cleveland) dataset
- 2 Download the dataset and load it into a working notebook environment
- 3 Understand every column: patient demographics, vitals, and cardiac test results
- 4 Identify the target column — presence (1) or absence (0) of heart disease
- 5 Check the number of rows (patient records) and columns (features)
- 6 Note the feature types: 13 clinical attributes, e.g. age, sex, chest pain type, resting BP, cholesterol, max heart rate

Data Understanding

Baseline profile of the Cleveland-style heart disease dataset before cleaning.



Number of records

303 patient records in the base dataset



Number of features

13 clinical input features + 1 target column



Data types

Mix of numeric (age, cholesterol) and categorical (chest pain type, thal)



Missing values

A small number of nulls in 'ca' and 'thal' columns



Duplicate values

Checked and removed to avoid data leakage



Unique values

Reviewed per categorical column to validate coded ranges



Statistical summary

Mean, median, std-dev and range computed for every numeric feature



Feature distribution

Age, cholesterol and heart rate reviewed for skew and spread

Data Preprocessing

Every check applied before the data is fit to train a model.



Missing values

Impute or drop nulls in 'ca' and 'thal' using median / mode strategies



Duplicate records

Detect and drop exact duplicate patient rows



Outliers

Flag extreme cholesterol / resting BP values with IQR or z-score checks



Incorrect data

Validate ranges (e.g. age > 0, valid coded categories)



Categorical variables

One-hot / label encode chest pain type, thal, slope, sex



Numerical variables

Standardize continuous features so Ridge/Lasso penalties act fairly

Visualizing the Data

Six chart types used to understand patterns before modelling.



Histogram

Distribution of age, cholesterol and resting blood pressure



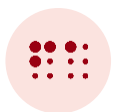
Bar chart

Counts of heart disease across categorical features



Box plot

Spread and outliers in cholesterol and max heart rate



Scatter plot

Relationship between age and max heart rate achieved



Correlation heatmap

Pairwise correlation across all clinical features



Count plot

Class balance between disease-present and disease-absent cases

Feature Selection & Engineering



Features used

All 13 clinical attributes — age, sex, chest pain type, resting BP, cholesterol, fasting blood sugar, ECG, max heart rate, exercise angina, ST depression, slope, vessels, thal.



Features removed

Highly redundant or near-zero-variance columns identified during correlation review are dropped to reduce noise.



Why removed

Reducing multicollinearity keeps Ridge/Lasso coefficients stable and easier to interpret clinically.



New features created

Optional binned age groups and interaction terms (e.g. age × cholesterol) tested to capture non-linear risk patterns.



Lasso as a built-in feature selector

L1 regularization shrinks the coefficients of weak predictors all the way to zero — automatically discarding features that add little diagnostic value, while Ridge keeps every feature but dampens their influence.

Train-Test Split

80%

20%



Training data (80%)

Used to fit the Logistic Regression, Ridge and Lasso model coefficients.



Testing data (20%)

Held out and untouched during training — used only to evaluate final performance.

A stratified split preserves the same ratio of disease-present to disease-absent cases in both the training and testing sets, so evaluation metrics stay reliable.

Model Building

Three logistic regression variants trained and compared on the same split.



Logistic Regression

Baseline classifier

No penalty on coefficients —
a reference point for how
much regularization helps.



Ridge (L2)

Penalizes large coefficients

Shrinks all coefficients
smoothly, reducing overfitting
while keeping every feature.



Lasso (L1)

Sparse, interpretable model

Drives weak feature
coefficients to exactly zero —
built-in feature selection.

Model Evaluation

Classification metrics used to compare the three models fairly.



Accuracy

Overall share of correctly classified patients



Precision

Of predicted-positive cases, how many truly have heart disease



Recall

Of actual positive cases, how many the model correctly catches



F1-score

Harmonic balance between precision and recall



Confusion Matrix

Full breakdown of true/false positives and negatives



ROC-AUC

How well the model separates disease from no-disease across thresholds



Save the Final Model

- 1 Select the best-performing model (Ridge or Lasso) based on validation metrics
- 2 Serialize the trained model and preprocessing pipeline using joblib / pickle
- 3 Store the saved model file alongside a short model card describing metrics and inputs
- 4 Version the artifact so future retraining never silently overwrites a working model

Deployment



Wrap the model

Load the saved model inside a lightweight Flask / Streamlit application



Expose an API

Serve predictions through a simple REST endpoint for other systems to call



Build a clinician UI

A short form for patient vitals, returning a risk prediction instantly



Host & monitor

Deploy to a cloud host and track prediction quality over time



Thank You

Heart Disease Prediction using Ridge- and Lasso-Regularized Logistic Regression